Universität Rostock



AGENTISCHES PROGRAMMIEREN

Joern Ploennigs · AI4SC

Midjourney: The engineer as architect and verifier, ref. The Great Wave

Hörsaalfrage

Der Code läuft ohne Fehlermeldung und liefert 2,8·10⁻⁵ N/mm². Stimmt das?

Prompt: „Berechne die Biegespannung eines Rechteckträgers."

Erzeugter Code:

```
b, h = 200, 400      # Breite, Höhe in mm
M = 150              # Moment in kNm
W = b * h**2 / 6     # Widerstandsmoment in mm³
sigma = M / W        # Spannung in N/mm²
print(f"Ergebnis: {sigma:.1e}")
```

Ergebnis des fehlerfreien Laufs:

```
Ergebnis: 2,8·10⁻⁵
```

Fehlerfrei ausgeführt heisst nicht richtig gerechnet

Prompt: „Berechne die Biegespannung eines Rechteckträgers.“

```python
b, h = 200, 400      # Breite, Höhe in mm
M = 150              # Moment in kNm
W = b * h**2 / 6     # Widerstandsmoment in mm³
sigma = M / W        # Spannung in N/mm²
print(f"Ergebnis: {sigma:.1e}")
```

```
Ergebnis: 2,8·10⁻⁵
```

Was sehen Sie, wenn Sie den Code lesen?

○ Kein Fehler, das Programm läuft durch

– Moment in kNm, Widerstandsmoment in mm³

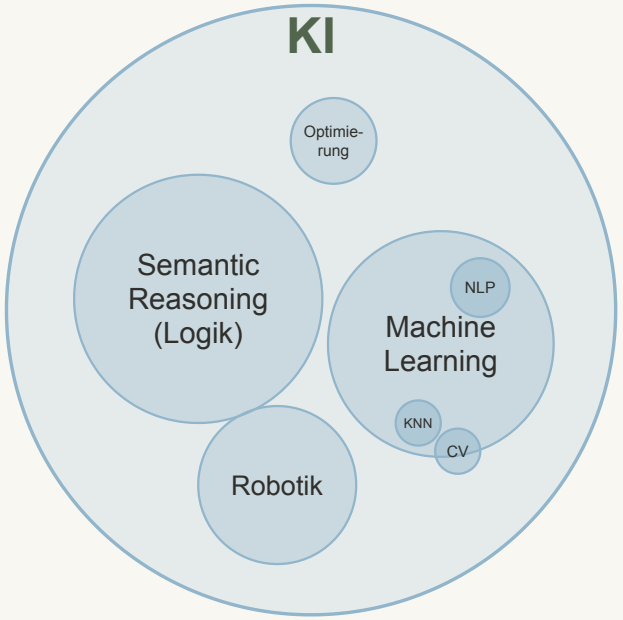
– Es fehlt der Faktor 10^6 für die Einheiten

\+ Richtig: 28,125 N/mm², nicht 0,000028

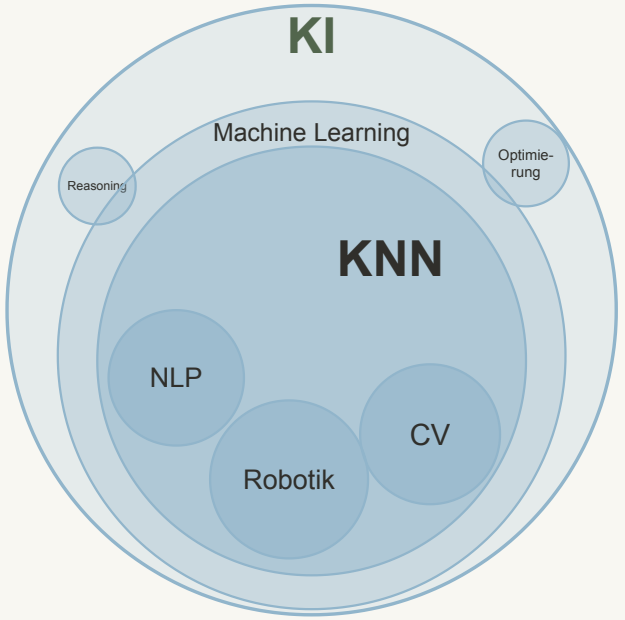
Warum macht ein so leistungsfähiges System so einen Fehler?

Gebiete der Künstlichen Intelligenz

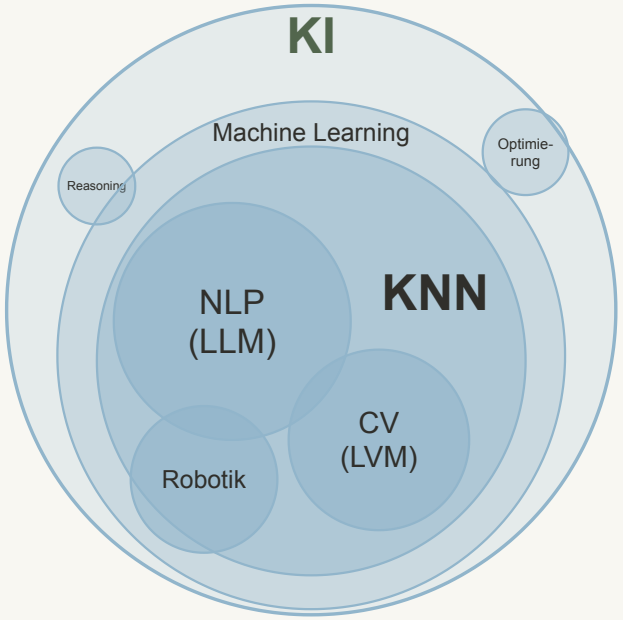
Semantic and Machine
Learning Age (2012)



Neural Network Age (2018)



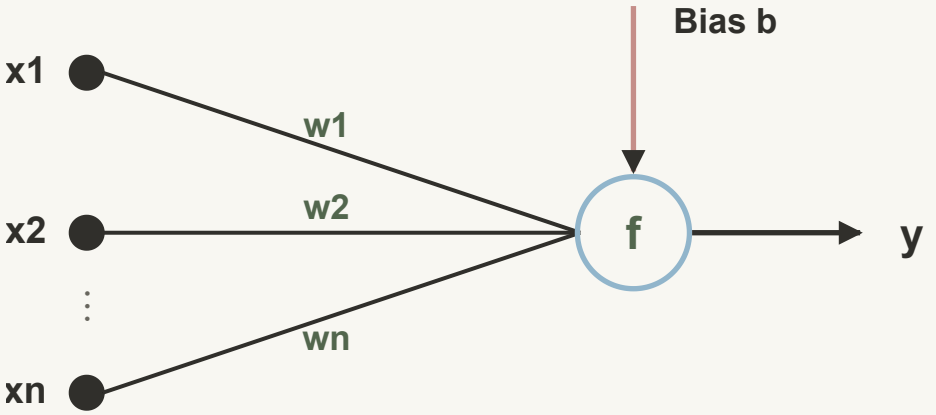
Large Foundation Model Age (2022)
(a.k.a. Generative Models)



- KNN – Künstliche Neuronale Netzwerke
- CV – Computer Vision (Bildverarbeitung) und Large Vision Models (LVM)
- NLP – Natural Language Processing (Sprachverarbeitung) und Large Language Models (LLM)

KNN – Künstliche Neuronale Netzwerke (1943)

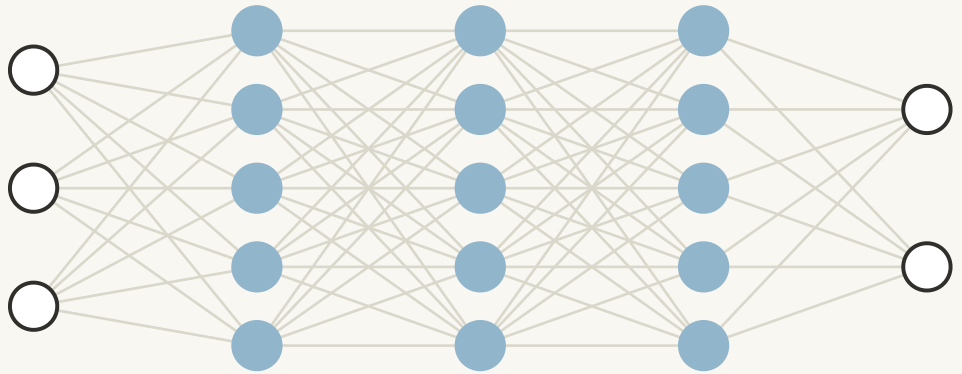
- Mathematische Nachbildung der Neuronen im menschlichen Gehirn
- Ein Neuronales Netzwerk setzt sich aus (sehr) vielen Neuronen zusammen
- Jedes Neuron ist meist einfach aufgebaut
  - Eingänge $x_1, x_2, \ldots, x_n$ die multipliziert werden mit den
  - Gewichten $w_1, w_2, \ldots, w_n$ und dazu wird ein
  - Bias b addiert
- Der Ausgang y ist die Summe davon, gefiltert mit einer Aktivierungsfunktion $f(\cdot)$ (Wir gewichten also die Eingänge nach „Wichtigkeit“)
- Die Aufgabe beim Lernen ist die Gewichte $w_1, w_2, \ldots, w_n$ und das Bias b zu bestimmen (Wir lernen welche Eingänge wichtig sind)
- **Der Trick:** Neuronale Netzwerke bestehen aus einfachen Bausteinen (→ einfach hoch-parallel zu berechnen)



$$y = f\left(b + \sum_{i=1}^{n} w_i x_i\right)$$

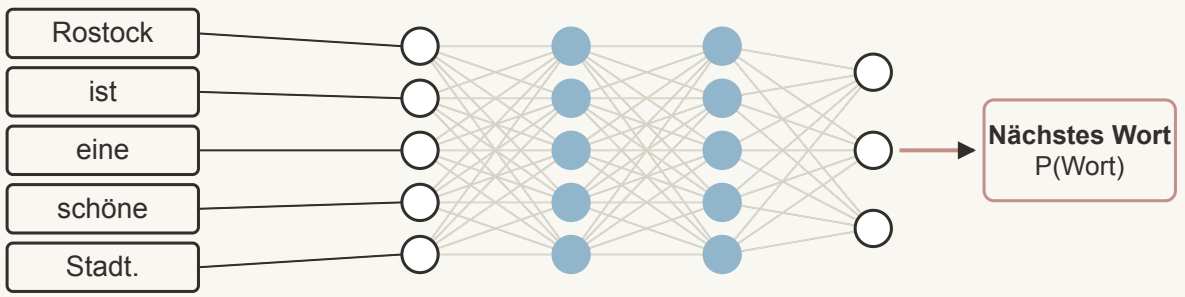
DNN – Tiefe Neuronale Netzwerke (seit 2011 skalierbar)

- Tiefe Neuronale Netzwerke enthalten sehr viele (3-150) tiefe Schichten von Neuronen
- Mit jeder Schicht steigt die Anzahl der Parameter multiplikativ und man erhält tausende Parameter
- Ab 2011 führten zwei Entwicklungen zur Berechenbarkeit
  - Parallelisierung der Berechnung in Graphikkarten mit tausenden von Recheneinheiten
  - Bessere Algorithmen, um die sehr große Anzahl an Parametern schnell zu bestimmen
- **Der Trick:** Mit den tieferen Schichten kann das Neuronale Netzwerk Wissen besser
  - **Abstrahieren** – da wir Eingänge immer mehr zusammen führen
  - **Verknüpfen** – Mit jeder Schicht verknüpfen wir das abstrahierte Wissen besser und betonen das Wichtige



Grosse Sprachmodelle (Large Language Models LLM) (seit 2017)

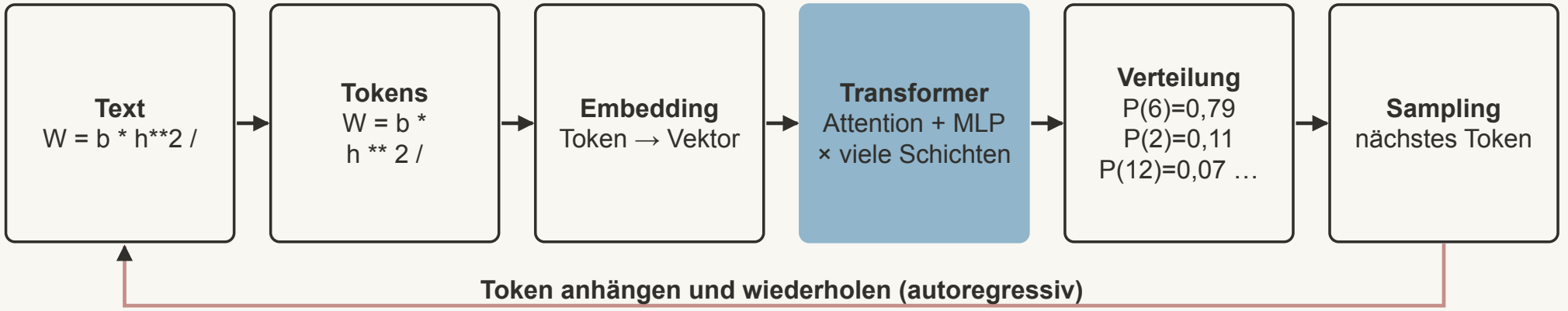
- Erhöhen die Anzahl der Parameter auf mehrere Milliarden
- Die Netzwerke zerlegen Texte in Worte und lernen mit welcher Wahrscheinlichkeit diese Worte im Text aufeinander folgen
- Mit der Entdeckung von Transformer Netzwerken (2017) konnte man diese Textzusammenhänge lernen
- **Der Trick:** dass das Netzwerk lernt
  - **Abstrahieren** – was sind die wichtigen Worte in einem Text
  - **Verknüpfen** – welche Worte sind mit diesen wichtigen Worten über Sätze verknüpft
- Wenn wir eine Anfrage an ChatGPT stellen, so wird das umgekehrt und ChatGPT generiert die wahrscheinliche Wortfolge für meine Anfrage



Eingabe: Wörter als Zahlen

Vaswani et al.: Attention Is All You Need, NeurIPS, 2017.

Ein Sprachmodell bildet Token-Folgen auf Wahrscheinlichkeiten ab



Attention: Jedes Token gewichtet alle vorherigen Tokens nach Relevanz.

Das Training belohnt Plausibilität, nicht Korrektheit

Training

01 Pre-Training: Fortsetzung von Text und Code vorhersagen

02 Instruction-Tuning: Aufgaben und Antworten nachahmen

03 Feedback: Antworten bevorzugen, die Menschen besser bewerten

$$\mathrm{L}(\theta) = -\sum_{t} \log P_{\theta}(x_t \mid x_{<t})$$

Was dieses Signal nicht enthält:

- Keine Einheitenprüfung
- Kein Ergebnis einer Ausführung
- Keine Kenntnis Ihrer Anforderung

Das Modell verallgemeinert Muster. Es schlägt nichts nach und rechnet nichts nach.

Programm code garantieren Korrektheit, ein Sprachmodell nicht

Programmcode

- Eingabe: Daten
+ Gleiche Eingabe, gleiche Ausgabe
- Fehler: Fehlermeldung mit Zeile

Sprachmodell

- Eingabe: Prompt
– Keine formale Garantie für die Ausgabe
- Fehler: trotzdem eine plausible Ausgabe

Wer garantiert die Korrektheit? Bei einem LLM sind Sie es.

Warum nicht?

Wie ein Ingenieur denkt

01 Übersetzt ein Problem in ein **mentales Modell**

02 Analysiert darin abstrakte Zusammenhänge
(versteht **warum und wann** der Balken sich biegt)

03 Kann kausale Schlussfolgerungen **ableiten**

04 Kann prüfen, ob das Ergebnis physikalisch sinnvoll ist (**kann rechnen**)

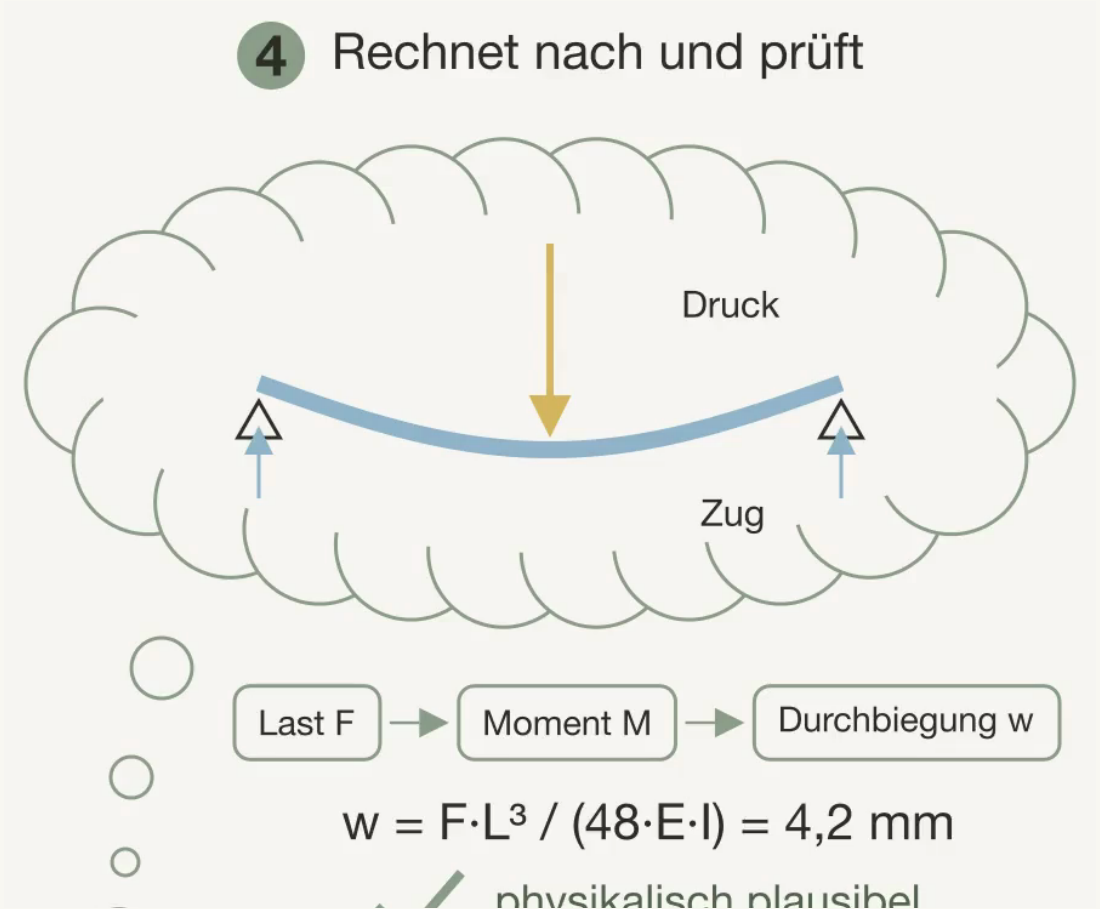
Wie ein LLM „denkt"

01 Übersetzt ein Problem in ein **sprachliches Muster**

02 Hat **kein** mentales Modell abstrakter Zusammenhänge
(weiß nur, dass „Balken" und „biegen" korrelieren)

03 Sagt **nur** statistisch wahrscheinliche Wörter voraus

04 Kann nicht prüfen, ob der Output sinnvoll ist (**kann nicht rechnen**)





Die Gefahren neuer Grundlagen KI-Modelle

Verstehen

- *Fehlendes räumliches und numerisches* Verständnis
- *Fehlendes logisches Erschließen* komplexer Zusammenhänge obwohl einfache funktionieren
- *Auslassen* statistisch irrelevanter aber fachlich wichtiger Details
- *Übersetzen* mit False-Friends, die in der Domäne andere Bedeutungen haben

Generieren

- *Halluzination* von falschen Inhalten
- *Fehlassistenz* durch fehlerhafte Ratschläge
- *Inkonsistenz* von generierten Inhalten (mehrere Sichten)
- *Bias* der Vorurteile/ Falschaussagen in den Trainingsdaten überbetont
- *Ineffizienz* durch generierte Texte, die keiner mehr liest
- *Fehloptimierungen* durch Lernen von Korrelationen in Trainingsdaten

Anpassen

- *Keine BIM-Unterstützung* zur Datenverarbeitung in KI-Modellen
- *Fehlende große Datensätze*
- *Unerklärliche* Black-Box-Modelle
- *Hohe Kosten* für regelmäßiges Training
- *Fehlendes Wissen* zu den Modellen

Kann KI das allein? → Die Kompetenzfalle

Verstehen

+ Zusammenfassen, Allgemeinwissen zu Bauen und Architektur
– Kein numerisches oder räumliches Verständnis
– Kein Detailwissen zur Baudomäne und zu Projektinformationen

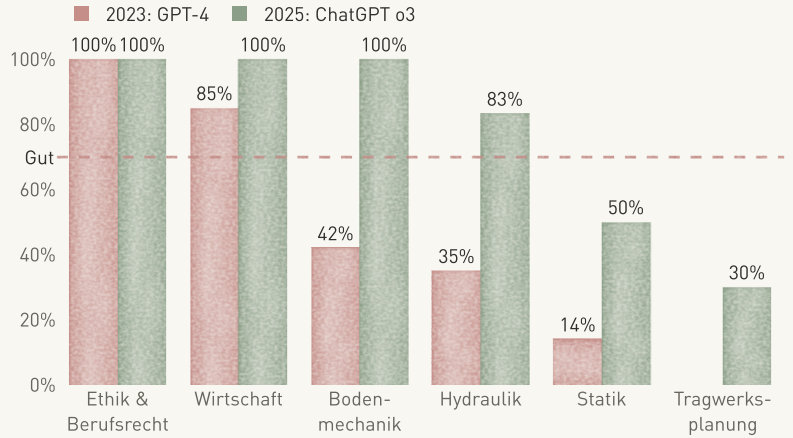
Generieren

+ Übersetzen, Bilder erzeugen
– Halluziniert bei Unwissenheit
– Erzeugt keine wiederholbar zusammenhängenden Ergebnisse

Anpassen

+ Wissen erweiterbar (RAG), Tuning von Modellen auf Spezialwissen
– Dennoch nicht an komplexe Ingenieurprobleme anpassbar

LLMs in der Ingenieur-Grundlagenprüfung (FE)



Daten: Pursnani et al. [2023], Oblitas et al. [2025]. Statik und Tragwerke bleiben am schwächsten.

Pursnani et al.: *Performance of ChatGPT on the US fundamentals of engineering exam*, **Computers and Education: AI**, 2023.
Oblitas, Adeeb & Cruz-Noguez: *Performance of AI Chatbots on the Fundamentals of Engineering Civil Exam*, **Discover Artificial Intelligence**, 2025.

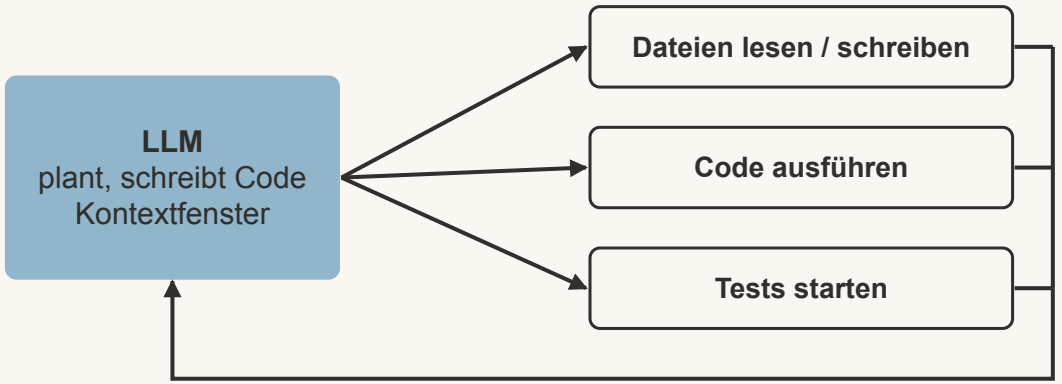
AGENTENISCHE PROGRAMMIERUNG

Code-Vervollständigung wurde in acht Jahren zum Agenten

2017	2020	2021	2022	2025
Transformer	GPT-3	Codex und Copilot	ChatGPT	Coding-Agenten
`Attention ersetzt Rekurrenz`	`175 Mrd. Parameter, Few-Shot-Lernen`	`LLM auf Code feinjustiert`	`Instruction-Tuning und menschliches Feedback`	`LLM mit Zugriff auf Dateien und Terminal`

Vaswani et al.: Attention Is All You Need, NeurIPS, 2017.
Brown et al.: Language Models are Few-Shot Learners, NeurIPS, 2020.
Chen et al.: Evaluating Large Language Models Trained on Code, 2021.
Ouyang et al.: Training language models to follow instructions with human feedback, 2022.

Ein Agent ist ein Sprachmodell in einer Schleife mit Werkzeugen

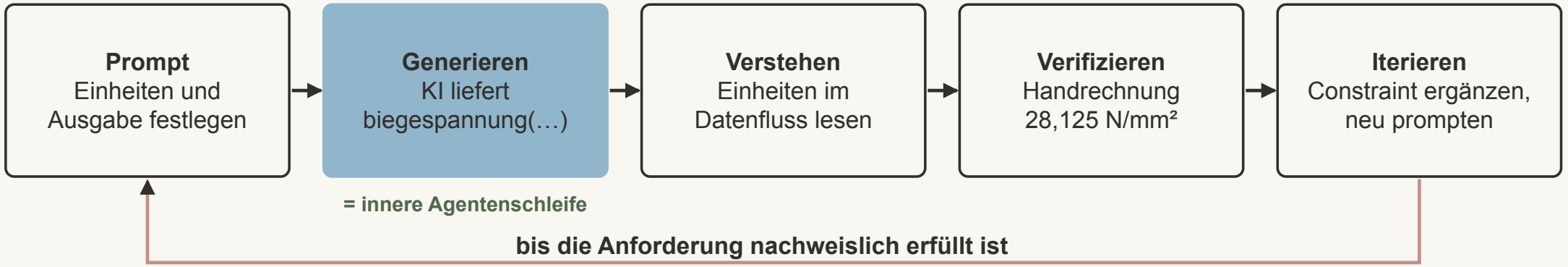


Der Agent schleift selbstständig.
Die Anforderung kennt nur der Mensch.

Autonomie der Werkzeuge:

- GitHub Copilot ergänzt Zeilen beim Tippen. Prüfen: jede Annahme per Tab
- Cursor bietet Chat und Edits im Editor. Prüfen: den Diff vor der Übernahme
- Claude Code plant, liest, führt aus und testet. Prüfen: das Ergebnis gegen die Anforderung

Der agentische Zyklus legt eine äussere Schleife um den Agenten



Jeder Schritt braucht eine eigene Kompetenz: Anforderungen formulieren, Code lesen, Testen denken.

Ein Prompt ist eine Anforderungsspezifikation

01 **Rolle:** Tragwerksplanerin, Python-Entwickler

02 **Kontext:** b, h in mm, M in kNm

03 **Aufgabe:** `biegespannung(b_mm, h_mm, M_knm)`

04 **Constraint:** intern in N und mm, keine Bibliotheken

05 **Ausgabeformat:** Docstring mit Einheiten, Typ-Hints

> „Du bist Python-Entwickler für Tragwerksplanung.
> Eingaben: b und h in mm, M in kNm.
> Schreibe `biegespannung(b_mm, h_mm, M_knm) -> float`.
> Rechne intern in N und mm, Ergebnis in N/mm².
> Keine externen Bibliotheken.
> Mit Docstring, der die Einheiten nennt, und Typ-Hints.“

Der fehlende Constraint verursacht den Einheitenfehler

Vager Prompt:

```python
def biegespannung_v1(b, h, M):
    W = b * h**2 / 6
    return M / W

print(f"{biegespannung_v1(200, 400, 150):.1e}")
```

Läuft, aber 10^6-fach zu klein

Prompt mit Einheiten-Constraint:

```python
def biegespannung(b_mm, h_mm, M_knm):
    """Biegespannung in N/mm²; M in kNm."""
    W_mm3 = b_mm * h_mm**2 / 6
    return M_knm * 1e6 / W_mm3

print(f"{biegespannung(200, 400, 150):.3f}")
```

Einheiten im Namen, Ergebnis stimmt

Lesen Sie zuerst den Vertrag der Funktion, dann die Implementierung

01 Signatur: Parameter, Einheiten, Rückgabetyp

02 Docstring: Was verspricht die Funktion?

03 Tests: Was ist das erwartete Verhalten?

04 Implementierung: Wie ist es umgesetzt?

Agentischer Ablauf

Wenn KI eine Funktion generiert:

01 Signatur lesen: Tragen die Namen die Einheiten?

02 Docstring prüfen: Stimmt die Einheit des Ergebnisses?

03 Testfall schreiben, bevor Sie die Implementierung lesen

04 Bei Abweichung: Prompt um den Constraint ergänzen

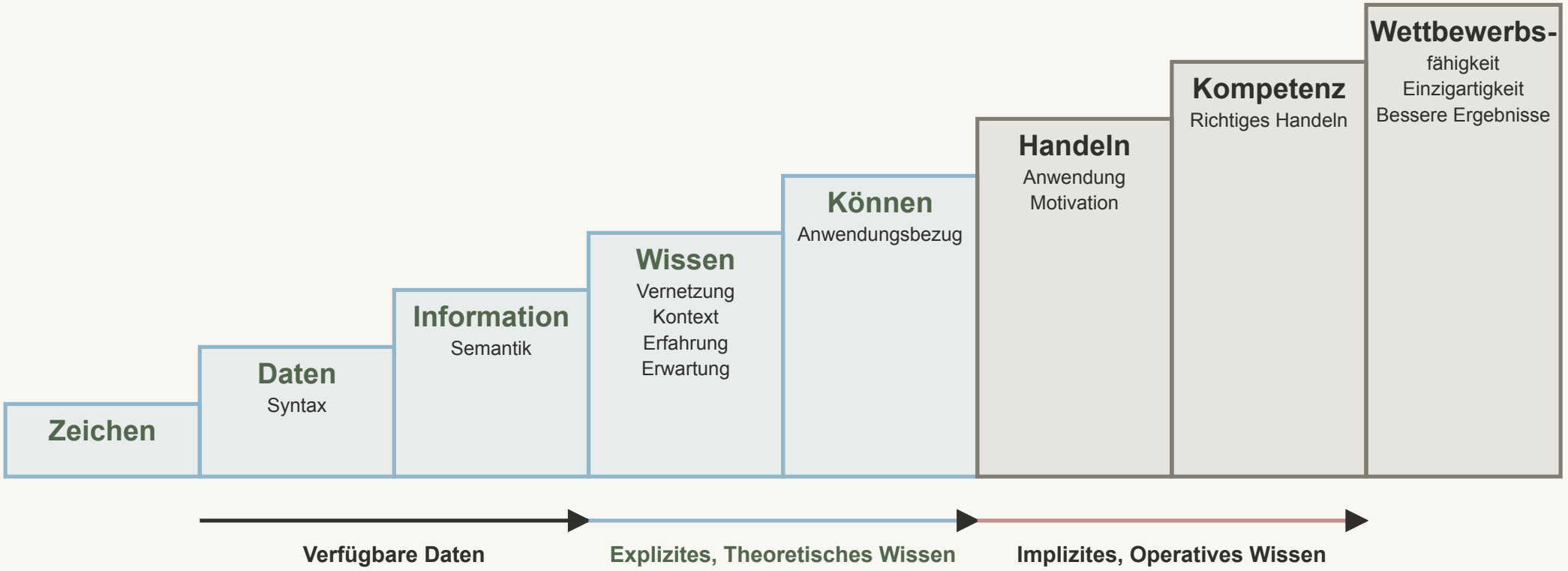
Ein Test mit Handrechnung findet den Fehler, bevor Sie lesen

```
erwartet = 28.125                    # Handrechnung in N/mm²
assert abs(biegespannung_v1(200, 400, 150) - erwartet) < 1e-3
```

```
assert abs(biegespannung(200, 400, 150) - erwartet) < 1e-3
print("Test bestanden")
```

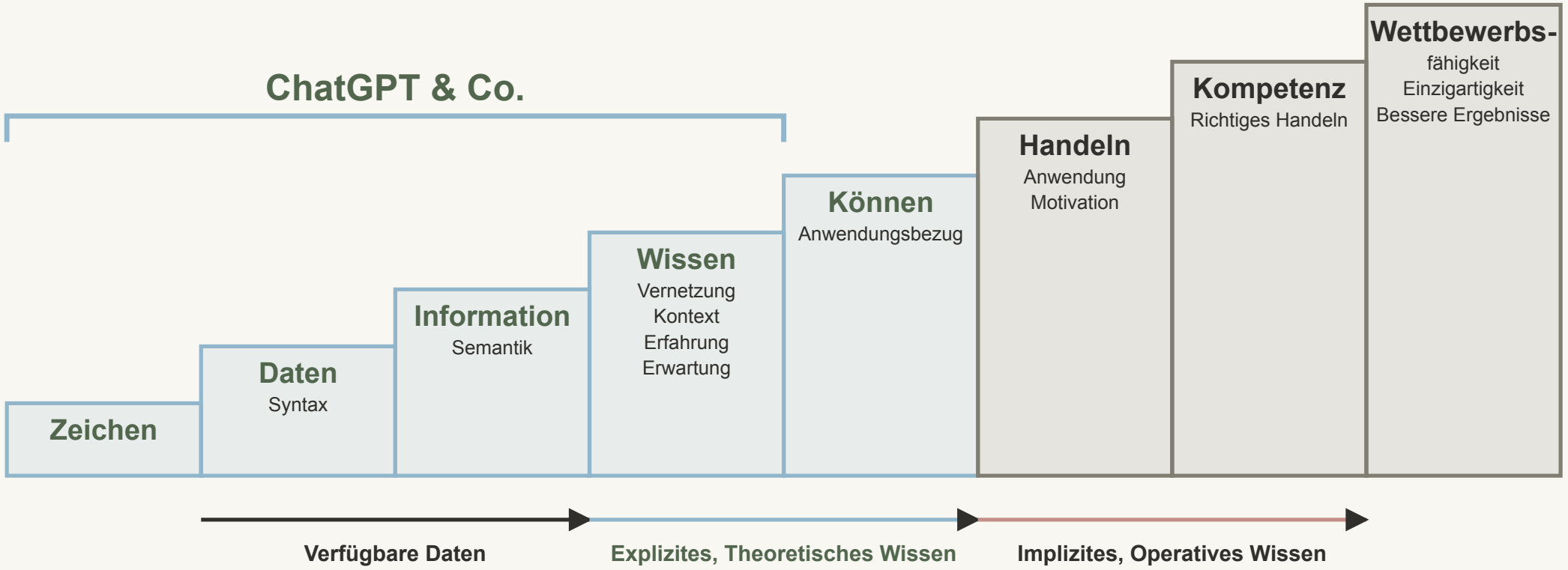
Eigener Test: Was passiert bei h = 0? Welches Verhalten verlangt die Anforderung?

Wissenstreppe



North, K. (2011): Wissensorientierte Unternehmensführung. Wertschöpfung durch Wissen.

Wissenstreppe und ChatGPT



North, K. (2011): Wissensorientierte Unternehmensführung. Wertschöpfung durch Wissen.

KI schreibt den Code, Sie verantworten ihn

Auftraggeber

- Anforderung präzise formulieren
- Im Beispiel: Einheiten fehlten im Prompt

Leser

- Code und Annahmen verstehen
- Im Beispiel: kNm durch mm^3 im Datenfluss

Verifizierer

- Ergebnis gegen die Anforderung prüfen
- Im Beispiel: Handrechnung ergibt 28,1

Die KI ist die Junior-Entwicklerin, Sie sind der Senior, der den Code abnimmt.

Vier Fehlerarten treten bei KI-Code immer wieder auf

1

Erfundene Funktion

`Bibliothek oder Methode existiert nicht`

2

Falsche Einheit

`kN statt N, mm statt m, wie im Beispiel`

3

Vergessener Randfall

`leere Liste, Division durch null`

4

Stilles Fehlverhalten

`läuft ohne Meldung, rechnet das Falsche`

Dann prüft eben der Mensch? → Die Kapazitätsfalle

Das Prüfen von KI-Output ist ermüdend ("AI Fatigue") und nicht trivial, Beispiel Softwareentwicklung:

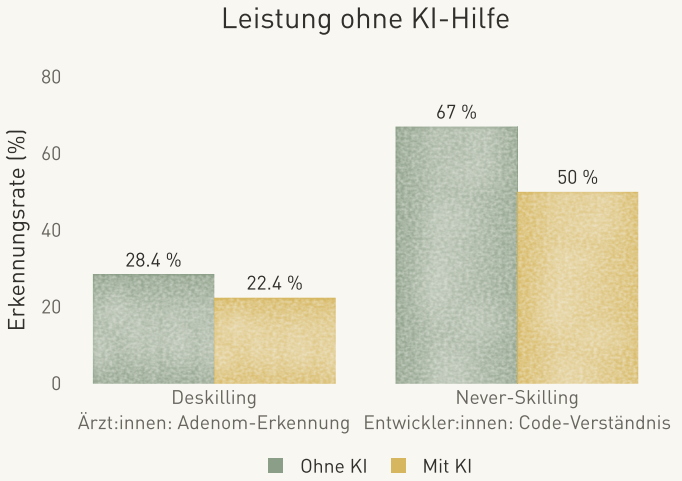
- KI-Output skaliert schneller als menschliche Review-Kapazität (4× mehr Pull Requests auf GitHub [CodeRabbit, 2026])
+ Die gefühlte Produktivität steigt [METR, 2025]
- Die gemessene Produktivität sinkt [METR, 2025]

> „Entwickler erwarteten, mit KI 24 % schneller zu sein, und selbst nachdem sie die Verlangsamung erlebt hatten, glaubten sie noch, KI habe sie um 20 % beschleunigt."
>
> — METR [2025]

METR: *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*, 2025.
CodeRabbit: *GitHub Gives Maintainers a Throttle for the AI Pull Request*, 2026.

Und wer kann morgen noch prüfen? → Die Fähigkeitsfalle

- Nutzer überschätzen ihre Fähigkeit, KI-Output zu prüfen [Fernandes et al., 2026]
- KI-Nutzung führt zu Cognitive Offloading und sinkendem kritischen Denken [Gerlich, 2025]
- → Deskilling: Erfahrene verlieren Fähigkeiten [Budzyń et al., 2025]
- → Never-Skilling: Einsteiger erlernen Grundlagen nicht [Ke et al., 2026]



Daten: Deskilling [Budzyń et al., 2025]; No-Skilling [Shen & Tamkin, 2026]

Teufelskreis **Deskilling**

Mehr KI-Assistenz
⌄
Cognitive Offloading
⌄
Deskilling
⌄
Never Skilling

Fernandes et al.: *AI makes you smarter but none the wiser*, **Computers in Human Behavior**, 2026.
Gerlich, M.: *AI Tools in Society: Impacts on Cognitive Offloading and the Future of Critical Thinking*, **Societies**, 2025.
Budzyń et al.: *Endoscopist deskilling risk after exposure to artificial intelligence in colonoscopy*, **Lancet Gastroenterology & Hepatology**, 2025.
Ke et al.: *AI-induced never-skilling in medical education*, **Nature Medicine**, 2026.

Was tun? → Den Teufelskreis durchbrechen

1. Bildung

- **Weiterbildung** für strukturierte KI-Nutzung
- **Ausbildung** für kritische Kompetenzen
- **Kultur** des Lernens und kritischen Denkens

2. Organisation

- **Prozesse** zur Prüfung von KI-Ergebnissen
- **Digitale Werkzeuge** zur Dokumentation im BIM
- **Führung**, die KI-Resilienz etabliert

3. Rahmenbedingungen

- **Forschung** zur Bewertung von KI-Resilienz
- **Regulierung** der Randbedingungen
- **Offenheit** für eine positive KI-Vision

Es braucht Zusammenarbeit, um Erfahrungen auszutauschen und Leitlinien zu entwickeln.

Der Zyklus ist die Methode des ganzen Kurses

Block	Thema	Trainierter Schritt
1	Computational Foundations	Verstehen
2	Computational Engineering	Prompt
3	Programming Foundations	Verstehen, Iterieren
4	Verification & Quality	Verifizieren
5	Problem Classes	Prompt, Verifizieren
6	Data Management	ganzer Zyklus

Jede Übung trainiert mindestens einen Schritt des Zyklus.

Referenzen

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020). Language Models are Few-Shot Learners. *NeurIPS*, 33, 1877–1901. https://arxiv.org/abs/2005.14165.

Budzyń, K., Romańczyk, M., Kitala, D., et al. (2025). Endoscopist deskilling risk after exposure to artificial intelligence in colonoscopy: a multicentre, observational study. *The Lancet Gastroenterology & Hepatology*, 10(10), 896–903. https://doi.org/10.1016/S2468-1253(25)00133-5.

Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. (2021). Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*. https://arxiv.org/abs/2107.03374.

CodeRabbit (2026). GitHub Gives Maintainers a Throttle for the AI Pull Request. https://www.coderabbit.ai/blog/github-gives-maintainers-a-throttle-for-the-ai-pull-request.

Fernandes, D., Kosch, T., & Woźniak, P. W. (2026). AI makes you smarter but none the wiser: The disconnect between performance and metacognition. *Computers in Human Behavior*. https://doi.org/10.1016/j.chb.2025.108779.

Gerlich, M. (2025). AI Tools in Society: Impacts on Cognitive Offloading and the Future of Critical Thinking. *Societies*, 15(1), 6. https://doi.org/10.3390/soc15010006.

Ke, Y., Jin, L., Ong, J. C. L., et al. (2026). AI-induced never-skilling in medical education. *Nature Medicine*, 32(6), 1997–2006. https://doi.org/10.1038/s41591-026-04438-y.

METR (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/.

North, K. (2011). *Wissensorientierte Unternehmensführung: Wertschöpfung durch Wissen*. Gabler.

Oblitas, L., Adeeb, S., & Cruz-Noguez, C. (2025). Performance of AI Chatbots on the Fundamentals of Engineering Civil Exam. *Discover Artificial Intelligence*, 5, 378. https://doi.org/10.1007/s44163-025-00646-6.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., et al. (2022). Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*. https://arxiv.org/abs/2203.02155.

Ploennigs, J., Berger, M., Wortmann, T., Kirchner, J., Beetz, J., Roitberg, A., et al. (2025). Building Foundation Models - Potentials, Challenges and Research Directions for Using LLM and LVM in AEC. *EC3*.

Pursnani, V., Sermet, Y., Kurt, M., & Demir, I. (2023). Performance of ChatGPT on the US fundamentals of engineering exam. *Computers and Education: AI*, 5, 100183. https://doi.org/https://doi.org/10.1016/j.caeai.2023.100183.

Shen, J. H., & Tamkin, A. (2026). How AI Impacts Skill Formation. *arXiv:2601.20245*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is All You Need. *NeurIPS*, 30. https://arxiv.org/abs/1706.03762.

QUESTIONS